DSU (gộp tập rời nhau)

Nhóm 3

03/2026

DSU là gì?

Khái niệm

DSU là viết tắt của Disjoint Set Union, còn gọi là Union-Find. Đây là cấu trúc dữ liệu dùng để quản lý các tập hợp rời nhau.

- Mỗi phần tử thuộc đúng một tập.
- Có thể kiểm tra hai phần tử có cùng tập hay không.
- Có thể gộp hai tập lại thành một tập mới.

DSU rất hữu ích khi xử lý thành phần liên thông và các thao tác gộp nhóm

Ý tưởng trực quan

- Ban đầu, mỗi đỉnh là một tập riêng biệt.
- Khi có quan hệ mới giữa hai đỉnh, ta có thể cần gộp hai tập của chúng.
- Nếu hai đỉnh đã ở cùng một tập, việc gộp lại là không cần thiết.

Ý tưởng trực quan

- Ban đầu, mỗi đỉnh là một tập riêng biệt.
- Khi có quan hệ mới giữa hai đỉnh, ta có thể cần gộp hai tập của chúng.
- Nếu hai đỉnh đã ở cùng một tập, việc gộp lại là không cần thiết.

Mục tiêu

DSU giúp duy trì các tập rời nhau một cách nhanh chóng, thay vì phải quét lại toàn bộ cấu trúc sau mỗi thao tác.

DSU dùng để làm gì?

- Quản lý thành phần liên thông khi thêm cạnh.
- Kiểm tra xem hai đỉnh có đang cùng một nhóm hay không.
- Phát hiện việc nối thêm một cạnh có tạo ra chu trình trong đồ thị vô hướng hay không.
- Giải các bài toán gộp nhóm theo kiểu động.

DSU dùng để làm gì?

- Quản lý thành phần liên thông khi thêm cạnh.
- Kiểm tra xem hai đỉnh có đang cùng một nhóm hay không.
- Phát hiện việc nối thêm một cạnh có tạo ra chu trình trong đồ thị vô hướng hay không.
- Giải các bài toán gộp nhóm theo kiểu động.

DSU là một trong những cấu trúc dữ liệu cơ bản nhất khi làm việc với đồ thị

Cấu trúc lưu trữ trong DSU

Hai mảng quan trọng

- `parent[x]`: cha của đỉnh `x` trong cây đại diện.
- `sz[x]` hoặc `rank[x]`: kích thước hoặc độ cao tương đối của cây.

- Nếu `parent[x] = x` thì `x` là gốc.
- Mỗi tập được biểu diễn bởi một cây.
- Gốc của cây là đại diện của cả tập.

Cấu trúc lưu trữ trong DSU

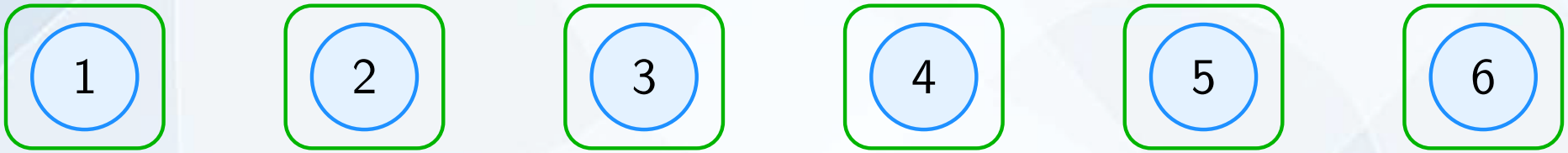
Hai mảng quan trọng

- `parent[x]`: cha của đỉnh `x` trong cây đại diện.
- `sz[x]` hoặc `rank[x]`: kích thước hoặc độ cao tương đối của cây.

- Nếu `parent[x] = x` thì `x` là gốc.
- Mỗi tập được biểu diễn bởi một cây.
- Gốc của cây là đại diện của cả tập.

Mỗi phần tử chỉ cần biết cha trực tiếp của nó

Ví dụ ban đầu



Ban đầu, mỗi đỉnh là một tập riêng: $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$.

Khởi tạo

Ý nghĩa

Mỗi phần tử bắt đầu là một tập riêng, nên:

```
parent[x] = x,     sz[x] = 1
```

- Không có phần tử nào nằm dưới phần tử khác.
- Mỗi cây lúc đầu chỉ có một nút.
- Đây là trạng thái khởi đầu chuẩn của DSU.

Khởi tạo

Ý nghĩa

Mỗi phần tử bắt đầu là một tập riêng, nên:

```
parent[x] = x,     sz[x] = 1
```

- Không có phần tử nào nằm dưới phần tử khác.
- Mỗi cây lúc đầu chỉ có một nút.
- Đây là trạng thái khởi đầu chuẩn của DSU.

Khởi tạo đúng là nền tảng để hai thao tác tìm gốc và gộp tập hoạt động chuẩn

Hàm `find(x)`

Mục đích

`find(x)` trả về đại diện của tập chứa phần tử `x`.

- Nếu `x` là gốc thì `find(x) = x`.
- Nếu chưa phải gốc, ta đi lên cha của nó cho đến khi gặp gốc.
- Kết quả cuối cùng là đại diện của toàn bộ tập chứa `x`.

Hàm `find(x)`

Mục đích

`find(x)` trả về đại diện của tập chứa phần tử `x`.

- Nếu `x` là gốc thì `find(x) = x`.
- Nếu chưa phải gốc, ta đi lên cha của nó cho đến khi gặp gốc.
- Kết quả cuối cùng là đại diện của toàn bộ tập chứa `x`.

Trực giác

Tìm gốc giống như lần theo các mũi tên trong một cây để tìm ra người đứng đầu.

Tối ưu hàm `find(x)`: nén đường đi

Nén đường đi

Sau khi tìm được gốc, ta cho các đỉnh trên đường đi trỏ thẳng về gốc. Điều này làm cho các lần truy vấn sau nhanh hơn rất nhiều.

- Lần đầu có thể đi qua nhiều đỉnh.
- Sau khi nén đường đi, cây trở nên phẳng hơn.
- Đây là một trong hai tối ưu quan trọng nhất của DSU.

Tối ưu hàm `find(x)`: nén đường đi

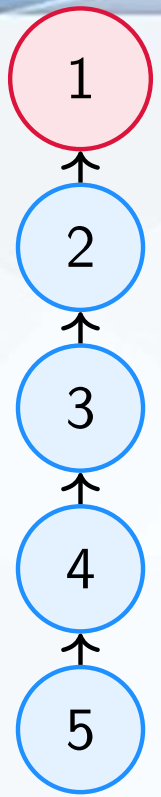
Nén đường đi

Sau khi tìm được gốc, ta cho các đỉnh trên đường đi trỏ thẳng về gốc. Điều này làm cho các lần truy vấn sau nhanh hơn rất nhiều.

- Lần đầu có thể đi qua nhiều đỉnh.
- Sau khi nén đường đi, cây trở nên phẳng hơn.
- Đây là một trong hai tối ưu quan trọng nhất của DSU.

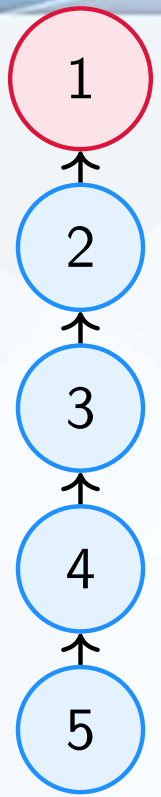
Nén đường đi giúp DSU chạy rất nhanh trong thực tế

Ví dụ nén đường đi



Trước khi nén, muốn tìm gốc của 5 ta phải đi qua 4, 3, 2, 1.

Ví dụ nén đường đi



Trước khi nén, muốn tìm gốc của 5 ta phải đi qua 4, 3, 2, 1.

Sau khi gọi `find(5)`

Các đỉnh trên đường đi có thể được cập nhật để trỏ trực tiếp về gốc 1.

Hàm `union(a,b)`

Mục đích

`union(a,b)` gộp hai tập chứa `a` và `b` lại với nhau.

- Tìm đại diện của hai tập: $u = \texttt{find(a)}$, $v = \texttt{find(b)}$.
- Nếu $u = v$ thì hai phần tử đã cùng một tập.
- Nếu khác nhau, nối một gốc vào gốc còn lại.

Hàm `union(a,b)`

Mục đích

`union(a,b)` gộp hai tập chứa `a` và `b` lại với nhau.

- Tìm đại diện của hai tập: $u = \texttt{find(a)}$, $v = \texttt{find(b)}$.
- Nếu $u = v$ thì hai phần tử đã cùng một tập.
- Nếu khác nhau, nối một gốc vào gốc còn lại.

Nguyên tắc

Không nối tùy ý, mà nên nối cây nhỏ vào cây lớn để giảm độ cao của cấu trúc.

Tối ưu `union`: gộp theo kích thước hoặc hạng

- Gộp theo kích thước: luôn nối cây nhỏ hơn vào cây lớn hơn.
- Gộp theo hạng: luôn nối cây có độ cao thấp hơn vào cây cao hơn.

Tối ưu `union`: gộp theo kích thước hoặc hạng

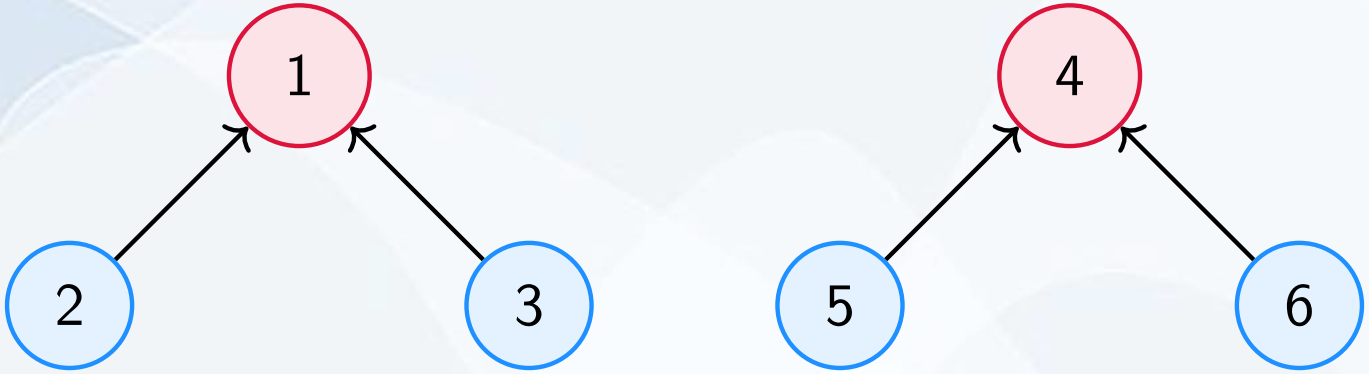
- Gộp theo kích thước: luôn nối cây nhỏ hơn vào cây lớn hơn.
- Gộp theo hạng: luôn nối cây có độ cao thấp hơn vào cây cao hơn.

Lợi ích

Hai kỹ thuật này giúp cây không bị lệch quá nhiều, từ đó giữ cho DSU rất nhanh.

Kết hợp nén đường đi + gộp theo kích thước/hạng là chuẩn tối ưu của DSU

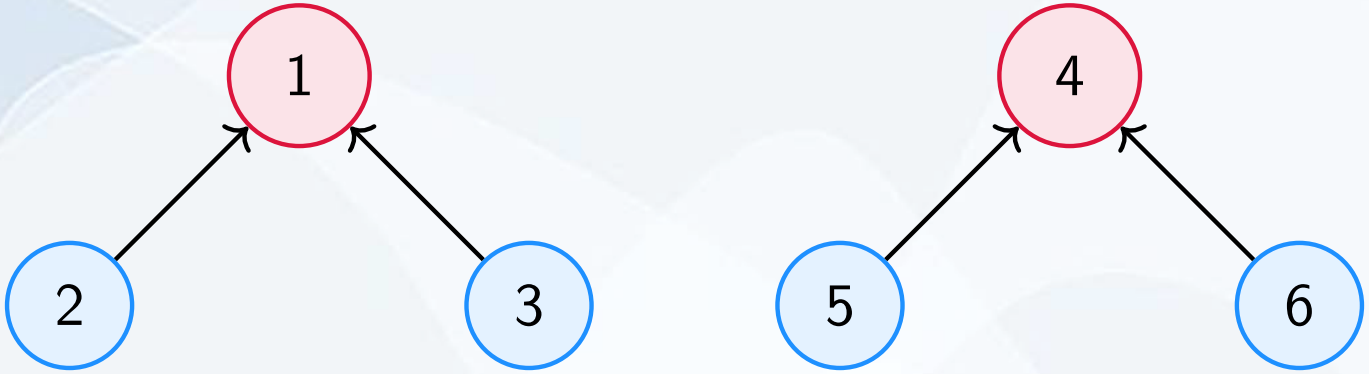
Minh họa gộp hai tập



Trạng thái trước khi gộp

Ta có hai tập: $\{1, 2, 3\}$ và $\{4, 5, 6\}$.

Minh họa gộp hai tập



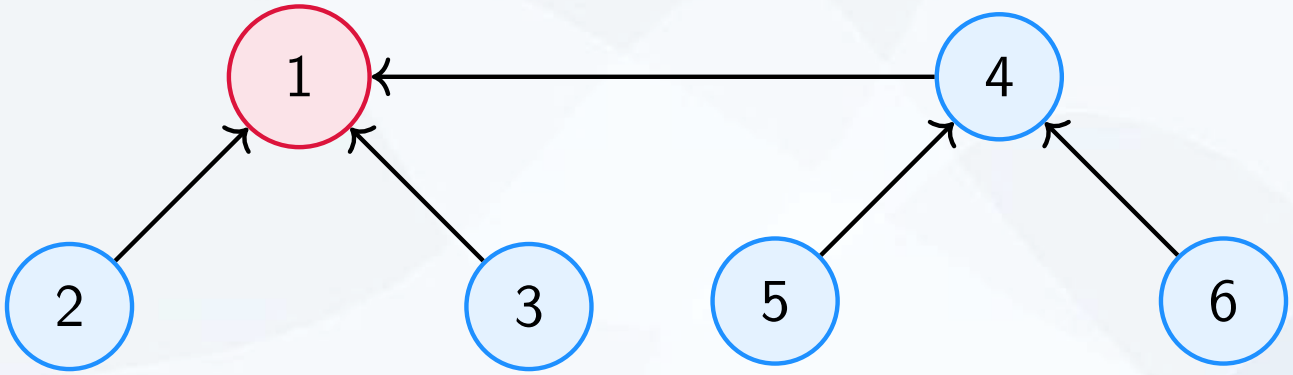
Trạng thái trước khi gộp

Ta có hai tập: $\{1, 2, 3\}$ và $\{4, 5, 6\}$.

Sau khi gộp `union(1,4)`

Chỉ cần nối một gốc vào gốc còn lại, hai tập sẽ nhập thành một.

Trạng thái sau khi gộp



- Gốc của tập mới là 1.
- Các phần tử 2,3,4,5,6 đều thuộc cùng một thành phần theo DSU.

Ví dụ mảng parent và sz

Giả sử

Ta có 6 đỉnh, sau một số lần gộp:

x	1	2	3	4	5	6
parent[x]	1	1	1	4	4	4
sz[x]	3	-	-	3	-	-

Ví dụ mảng parent và sz

Giả sử

Ta có 6 đỉnh, sau một số lần gộp:

x	1	2	3	4	5	6
parent[x]	1	1	1	4	4	4
sz[x]	3	-	-	3	-	-

- Tập thứ nhất có gốc 1, gồm $\{1, 2, 3\}$.
- Tập thứ hai có gốc 4, gồm $\{4, 5, 6\}$.
- Nếu gộp hai tập này, chỉ cần nối một gốc vào gốc kia.

Kiểm tra hai phần tử có cùng tập hay không

Quy tắc

Hai phần tử a và b thuộc cùng một tập khi và chỉ khi:

```
find(a) = find(b)
```

- Nếu kết quả bằng nhau, hai đỉnh đã được nối gián tiếp với nhau.
- Nếu khác nhau, chúng nằm ở hai thành phần khác nhau.

Kiểm tra hai phần tử có cùng tập hay không

Quy tắc

Hai phần tử a và b thuộc cùng một tập khi và chỉ khi:

```
find(a) = find(b)
```

- Nếu kết quả bằng nhau, hai đỉnh đã được nối gián tiếp với nhau.
- Nếu khác nhau, chúng nằm ở hai thành phần khác nhau.

Đây là thao tác nền tảng để trả lời nhiều truy vấn trên đồ thị

Ví dụ kiểm tra cùng tập

Giả sử

Sau một số phép gộp, ta có:

$$\{1, 2, 3\}, \quad \{4, 5\}, \quad \{6\}$$

- `find(2) = find(3)` $\rightarrow$ cùng tập.
- `find(2)` $\neq$ `find(5)` $\rightarrow$ khác tập.
- `find(6) = 6` $\rightarrow$ đỉnh 6 đang là gốc của một tập riêng.

Ví dụ kiểm tra cùng tập

Giả sử

Sau một số phép gộp, ta có:

$$\{1, 2, 3\}, \quad \{4, 5\}, \quad \{6\}$$

- `find(2) = find(3)` $\rightarrow$ cùng tập.
- `find(2)` $\neq$ `find(5)` $\rightarrow$ khác tập.
- `find(6) = 6` $\rightarrow$ đỉnh 6 đang là gốc của một tập riêng.

DSU trả lời các truy vấn cùng nhóm rất nhanh

Mã giả DSU

```
make_set(x):
    parent[x] = x
    sz[x] = 1
find(x):
    if parent[x] == x:
        return x
    parent[x] = find(parent[x])
    return parent[x]
union(a, b):
    a = find(a)
    b = find(b)
    if a == b:
        return
    if sz[a] < sz[b]:
        swap(a, b)
    parent[b] = a
    sz[a] += sz[b]
```

Giải thích mã giả

- `make_set`: tạo một tập mới chỉ chứa một phần tử.
- `find`: đi lên gốc và nén đường đi.
- `union`: tìm gốc của hai tập, nếu khác nhau thì gộp lại.

Giải thích mã giả

- `make_set`: tạo một tập mới chỉ chứa một phần tử.
- `find`: đi lên gốc và nén đường đi.
- `union`: tìm gốc của hai tập, nếu khác nhau thì gộp lại.

Điểm quan trọng

Dòng

```
parent[x] = find(parent[x])
```

chính là bước nén đường đi.

Ví dụ tuần tự các thao tác

Ban đầu

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}$$

1. `union(1,2)` $\rightarrow \{1,2\}, \{3\}, \{4\}, \{5\}$
2. `union(3,4)` $\rightarrow \{1,2\}, \{3,4\}, \{5\}$
3. `union(2,3)` $\rightarrow \{1,2,3,4\}, \{5\}$
4. `find(4)` $\rightarrow$ trả về đại diện của tập chứa 4

Ví dụ tuần tự các thao tác

Ban đầu

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}$$

1. `union(1,2)` $\rightarrow \{1,2\}, \{3\}, \{4\}, \{5\}$
2. `union(3,4)` $\rightarrow \{1,2\}, \{3,4\}, \{5\}$
3. `union(2,3)` $\rightarrow \{1,2,3,4\}, \{5\}$
4. `find(4)` $\rightarrow$ trả về đại diện của tập chứa 4

Qua mỗi bước, số lượng tập giảm dần khi ta gộp các thành phần

Độ phức tạp

Khi tối ưu đầy đủ

Với nén đường đi và gộp theo kích thước/hạng, DSU hoạt động rất nhanh trong thực tế.

- `find`: gần như $O(1)$ trung bình.
- `union`: gần như $O(1)$ trung bình.
- Phân tích chặt hơn cho kết quả $O(\alpha(n))$, với $\alpha(n)$ là hàm nghịch đảo Ackermann.

Độ phức tạp

Khi tối ưu đầy đủ

Với nén đường đi và gộp theo kích thước/hạng, DSU hoạt động rất nhanh trong thực tế.

- `find`: gần như $O(1)$ trung bình.
- `union`: gần như $O(1)$ trung bình.
- Phân tích chặt hơn cho kết quả $O(\alpha(n))$, với $\alpha(n)$ là hàm nghịch đảo Ackermann.

Trong các bài toán thực tế, DSU gần như rất gần hằng số

Ứng dụng của DSU trong lý thuyết đồ thị

- Theo dõi các thành phần liên thông khi đồ thị được xây dần.
- Kiểm tra một cạnh mới có làm hai đỉnh rơi vào cùng một nhóm hay không.
- Phân chia các đỉnh thành các cụm rời nhau.
- Giải các bài toán cần thao tác gộp nhóm liên tục.

Ứng dụng của DSU trong lý thuyết đồ thị

- Theo dõi các thành phần liên thông khi đồ thị được xây dần.
- Kiểm tra một cạnh mới có làm hai đỉnh rơi vào cùng một nhóm hay không.
- Phân chia các đỉnh thành các cụm rời nhau.
- Giải các bài toán cần thao tác gộp nhóm liên tục.

DSU đặc biệt phù hợp khi đồ thị được xét theo kiểu thêm cạnh dần dần

Tóm tắt DSU

- DSU quản lý các tập rời nhau.
- Có hai thao tác chính: tìm gốc và gộp tập.
- Nén đường đi và gộp theo kích thước/hạng giúp tối ưu mạnh.
- DSU rất hữu ích trong các bài toán đồ thị cần gộp nhóm và kiểm tra cùng thành phần.

Tóm tắt DSU

- DSU quản lý các tập rời nhau.
- Có hai thao tác chính: tìm gốc và gộp tập.
- Nén đường đi và gộp theo kích thước/hạng giúp tối ưu mạnh.
- DSU rất hữu ích trong các bài toán đồ thị cần gộp nhóm và kiểm tra cùng thành phần.

DSU là cấu trúc dữ liệu gọn, nhanh và rất quan trọng

Kết luận

- DSU là công cụ cơ bản để quản lý các thành phần rời nhau.
- Cách cài đặt đơn giản nhưng hiệu quả rất cao.
- Khi học lý thuyết đồ thị, DSU là một trong những cấu trúc dữ liệu nên nắm thật chắc.

Cảm ơn!